Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	12-10-2026
Team ID	SWUID2026403484
Project Name	HotelPro – Hotel Room Booking and Management System
Maximum Marks	4 Marks

Technical Architecture:

HotelPro is designed as a scalable, self-contained 3-tier web system: a React.js single-page application with Bootstrap (presentation tier), a stateless Node.js and Express.js REST API (application tier) and a MongoDB document database accessed through the Mongoose ODM (data tier). The client communicates with the API over HTTPS using JSON; authentication and authorisation middleware verifies signed JSON Web Tokens (JWT) on every protected request and enforces role-based access control (RBAC) across Customers, Hotel Staff and Administrators. External integrations are cleanly scoped to a certified third-party payment gateway (sandbox/live) and an optional SMTP email/SMS service, keeping the platform secure, modular and easy to extend.

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	**User Interface**	Responsive single-page web application for customers/guests, hotel staff/receptionists and administrators: home/landing search bar, register/login, room search results with filters/sorting, room details, booking & payment, My Bookings, Staff Console and Admin Dashboard	HTML5, CSS3, JavaScript (ES6+), React.js, React Router, Axios, Bootstrap
2.	**Application Logic-1**	User registration, login/logout, bcrypt password hashing, JWT session handling, RBAC middleware (customer, staff, admin), hotel/room inventory CRUD and room search/availability filtering	Node.js (LTS), Express.js, jsonwebtoken (JWT), bcrypt
3.	**Application Logic-2**	Room booking engine: real-time availability check, confirmation-time re-verification to prevent double booking, automatic nightly price + tax calculation, booking modification/cancellation and room date release	Node.js / Express.js controllers & services, Mongoose ODM validation

4.	**Application Logic-3**	Front-desk guest record management (ID proofs, accompanying guests), check-in/check-out processing with actual timestamps and room state updates, online/offline payment handling, invoicing and admin KPI analytics/CSV/PDF reports	Node.js / Express.js REST API, Payment Gateway API, Charting & CSV/PDF export utilities

S.No	Component	Description	Technology
5.	**Database**	Document-oriented NoSQL database storing users, hotels, rooms, bookings (with embedded guest arrays) and payments collections, enforced with Mongoose schemas and reference ObjectIds	MongoDB 6.0+ (local instance or MongoDB Atlas cloud cluster), Mongoose ODM
6.	**Database Indexing & Query Optimisation**	Fast sub-2-second search and double-booking prevention using unique index on users.email, index on hotels.city, unique index on payments.transactionId and compound index on bookings (roomId, checkInDate, checkOutDate)	MongoDB Indexes & Compound Indexing
7.	**Media & Asset Handling**	Hotel property photos, room gallery images (URLs), payment receipts/invoices and optional front-desk ID scanner/receipt printer integration via browser facilities	Image URLs / Cloud Storage, Browser Print & File-Upload APIs
8.	**Email / Notifications**	Transactional emails and alerts for booking confirmations, cancellation/refund updates and password reset links	SMTP / Transactional Email Service (optional)
9.	**External API**	Certified third-party payment gateway (sandbox for testing, live for production) supporting credit/debit cards, UPI and net banking; zero sensitive card/CVV data stored locally	Third-Party Payment Gateway REST API

10.	**Infrastructure & Runtime**	Client-server communication over HTTP/1.1+ secured with TLS (HTTPS), CORS restricted to approved front-end origins, environment variable secret management and cross-platform server hosting	Windows 10/11, Linux (Ubuntu 20.04+) or macOS, Node.js LTS, HTTPS/TLS

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
11.	**Open-Source Frameworks & Libraries**	Frontend UI, routing, HTTP client, backend web framework, ODM and authentication libraries	React.js, React Router, Bootstrap, Axios, Node.js, Express.js, Mongoose, jsonwebtoken, bcrypt
12.	**Security Implementations**	HTTPS/TLS encryption, salted bcrypt password hashing, stateless JWT Bearer tokens with configurable expiry (e.g., 1 hr) and refresh tokens, RBAC middleware (HTTP 401/403), NoSQL injection/XSS/CSRF sanitisation, auth rate limiting, CORS allow-list and zero card storage	JWT (jsonwebtoken), bcrypt, Express Auth/RBAC Middleware, Rate Limiting, CORS, HTTPS/TLS

S.No	Characteristics	Description	Technology
13.	**Scalable Architecture**	3-tier client-server architecture with a stateless JWT REST API for horizontal scaling behind a load balancer and MongoDB replica sets or sharded clusters	Modular MVC (Routes, Controllers, Models, Middleware, Services), REST/JSON, MongoDB Atlas
14.	**Availability & Performance**	99% target availability with automated DB backups; pages load within 3 s and 95% of API requests return within 2 s under normal load; supports 500+ concurrent users; confirmation-time availability re-verification prevents double booking	MongoDB Compound Indexing, Stateless Node.js Event Loop, Automated Backups

15.	**API Documentation**	RESTful JSON endpoints (GET, POST, PUT/PATCH, DELETE) using standard HTTP status codes (200, 201, 400, 401, 403, 404, 409, 500), documented and tested via collections	Postman Collections, REST/JSON Conventions
16.	**Testing & Quality Assurance**	Requirement-driven functional verification, REST API endpoint testing, role-based access validation, input sanitisation checks and cross-browser responsive UI testing	Postman, Browser DevTools (Chrome, Firefox, Edge, Safari)
17.	**Version Control & Tools**	Source code management with branches and pull requests, package management and IDE tooling	Git, GitHub, npm, Visual Studio Code

References:

React.js Documentation

Node.js & Express.js Documentation

OWASP Top 10

How to draw useful technical architecture diagrams